

先看懂：哪些 token 要重新计算？

从一个 10,000 token 的 prompt 开始。把 token 理解为模型处理文本的小单位即可。

你说的 miss：prefill 阶段，无法复用已有 KV、需要重新计算的 token 数量或比例。这里按“可复用前缀 + 新计算后缀”理解。[1][2]

教学例子：总长 10,000 token，miss 比例 20%



命中缓存，也可能需要读盘。命中表示“有可复用结果”；结果是否已经在 NPU 内存里，是另一个问题。

KV 是可复用的中间计算结果。NPU 负责计算；SSU 是存放 KV 的存储设备；I/O 是搬运数据。prefill 是处理输入、为生成答案做准备的阶段。

后面都用：L = 总 token 数；p = miss 比例；M = miss token 数。

$M = p \times L = NQL$ 。例如 $p = 20\% = 0.2$ 。

边界：若命中 KV 已驻留 NPU 内存，就不需要全部从 SSU 读取；本图不能直接套用。来源 [1][2] 见第 8 页。

为什么 miss 越少，搬运反而越吃紧？

比较同样长度的输入：少算了很多，仍要搬运大部分已有 KV。

固定总长和执行配置时，可先近似理解为：
计算时间 C ≈ 每 token 计算成本 × miss token 数
读取量 V = 每 token 的 KV 大小 × 命中 token 数

为把读取藏在一层计算里，需要的平均速率约为 $B = V / C$ 。若进一步把每 token 计算成本看作常数，就有 $B \propto (1 - p) / p$ 。p 越小，可供预取的计算时间越短。

当前 data：总长均为 32K = 32,768 token

miss 数 (NQL)	miss 比例	计算 C (ms / 层)	读取 V (MiB / 层)	需求 V/C (GiB/s)
128	0.3906%	1.178	43.828	36.333
512	1.5625%	3.702	43.312	11.425
2048	6.2500%	14.369	41.250	2.803
4096	12.5000%	28.593	38.500	1.315

看第一行与第三行：计算从 14.369 ms 缩到 1.178 ms；读取量仍为约 41 ~ 44 MiB。因此需求速率从 2.803 升到 36.333 GiB/s。

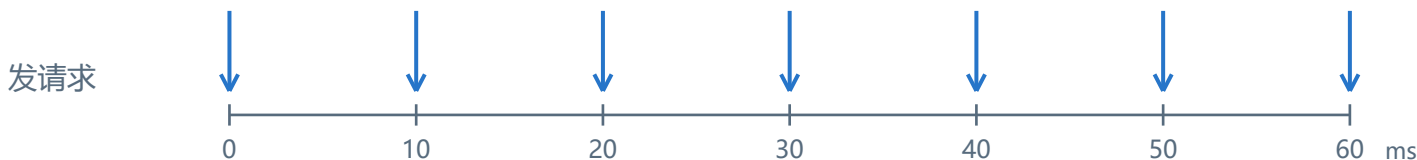
“正比”是近似，不是 data 的硬规则：miss 数翻 4 倍时，128→512 的计算时间只变为约 3.14 倍。上下文长度、固定开销、批处理与执行效率也会影响计算成本。

真实数值读取自仓库 data；C 与 V 分别输入仿真。GiB/s 为二进制带宽单位，1 GiB = 1,024 MiB。来源 [2][3]。

开环与闭环：下一次发送听谁的？

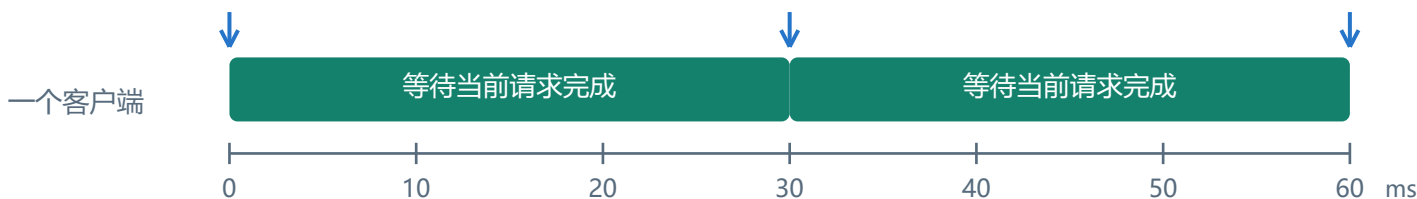
“开闭环”说的是发送规则；“有限”说的是总任务数量。它们是两个维度。

开环：按时间表发，前面的慢了也照发



例如每 10 ms 到达一个请求；即使处理一个要 30 ms，外部到达时间仍不变。

闭环：这个做完，再发下一个



同一个客户端：第 1 个用 30 ms，就在 30 ms 发第 2 个。若第 1 个变成 50 ms，第 2 个也随之推迟。

说法	可以怎样输入
开环有限请求	提前定好 1,000 个到达时间，发完后等待排空。

闭环有限请求 每个客户端做完一个再发一个，各自清单用完就结束。

我们的实验：外部有限请求先进入各卡队列；卡内执行和后续 I/O 受完成进度影响。arrival = 0 只代表请求已到等待队列，并不代表八层 I/O 都已发给 SSU。

开闭环的常见定义见 [4]；当前请求到达事件与完成驱动接纳见 [3]。有限闭环不等同于无限循环的稳态模型。

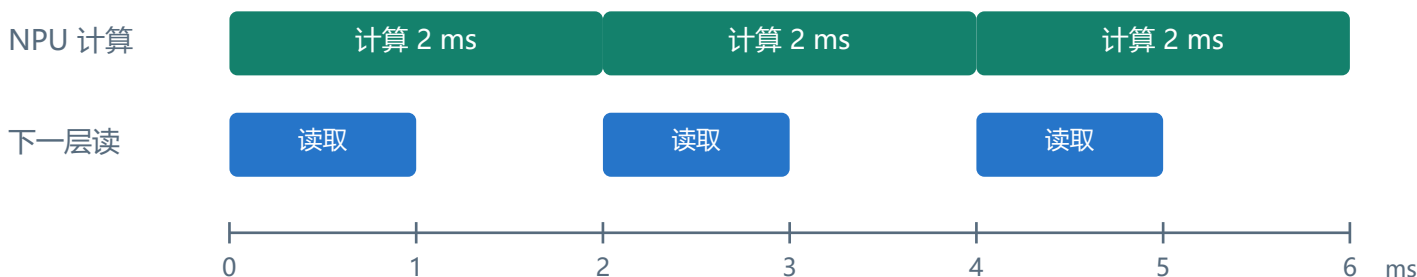
NPU 为什么会停下来等？

计算当前层时，提前读取下一层；下一层的数据到齐，才可以继续算。

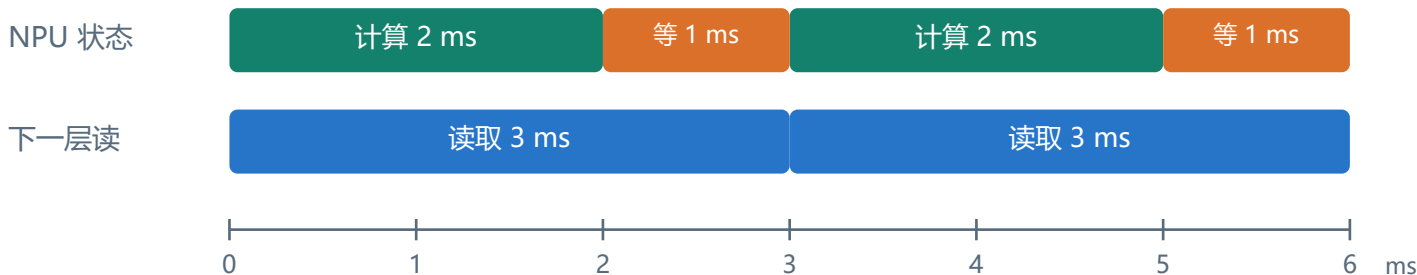
预取 = 提前搬下一层的数据。**I/O stall** = 算完了，但下一层数据还没到齐，只能等。

教学算例：每层计算 $C = 2\text{ ms}$ ；忽略启动与结束

A 读取只要 1 ms：全部藏在计算里



B 读取要 3 ms：每轮会露出 1 ms 等待



NPU 利用率 = 正在计算的时间 ÷ 观察时间

A: $6 / 6 = 100\%$ B: $4 / 6 \approx 66.7\%$

有稳定的每卡服务速率 b 、相同 C 和 V ，并且逐层预取时：

$U \approx C / \max(C, V/b)$ 。真实 FIFO 还要加上排队与接收链路的影响。

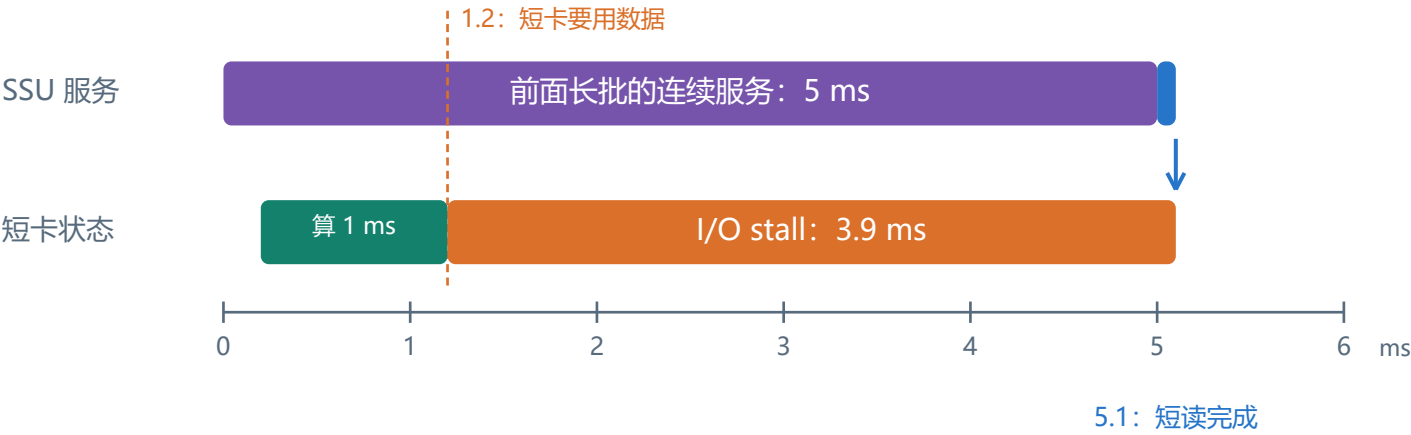
当前代码在本层开算时激活下一层读取；最后一层可预取已到达的下一请求首层。跨请求的预取预算来自前请求末层。来源 [3]。

一条 FIFO 队列，短读取怎样被挡住？

教学算例：只看 1 块 SSU 和 2 张 NPU，盘速率 40 GiB/s；忽略接收链路。

下一层读取	读取量 V	当前层计算 C	名义需求 V/C
长批读取	0.200 GiB	100 ms	2 GiB/s
短批读取	0.004 GiB	1 ms	4 GiB/s

t = 0：长卡开算，并已把下一层的长批数据排进 FIFO。
t = 0.2 ms：短卡开算并发出下一层读取；它希望在 **1.2 ms** 前拿到数据。



短读自身只需 0.1 ms，却等到 5.1 ms 才完成。
直接挡住它的是前面的读取工作；其他卡的长计算本身不占用这张短卡。

两卡名义需求合计 $2 + 4 = 6 \text{ GiB/s} < 40 \text{ GiB/s}$ ，仍可能发生这次等待。原因是数据成批排队，而短卡要求很快到齐。**V/C 欠载，不等于禁止瞬时突发。**

假设：长批已先入队，短读不能越过它。图中长条代表许多小 I/O 的累计服务，不是一条不可中断 5 ms 的命令。本例不能证明 32 卡长期下降 10%。

看带宽时，要分清三个数字

想验证“欠载下 FIFO 很差”，需要证明输入有余量，同时追踪数据是否按时到齐。

数字	它回答什么问题？
容量：每盘 40 GiB/s	这块盘最多能搬多快？6 块合计 240 GiB/s。
名义需求：V/C	如果希望读数藏在计算里，这个请求需要多快？
实际吞吐：已搬字节 / 时间	在这段时间里，盘实际上搬了多少数据？

实际吞吐低，不能独自证明输入欠载。
NPU 等待 → 后面的计算推迟 → 后续读取也推迟 → 盘收到的工作可能减少。这就是闭环会“自己减速”的原因。

逐盘检查：此前名义口径是对每块盘，累加**当前已接纳请求**的 V_s/C (V_s 是该盘读取量)，而非只与总容量 240 比较。跨请求首层预取的额外释放及前请求末层预算应另列；名义欠载不能保证每次读取都按时完成。

一个反例：第 2 页的 32K / NQL=128，每卡需求约 36.333 GiB/s。若 32 张卡同时运行这种画像，合计约 **1,162.65 GiB/s**，已远超 240。这种情况下利用率低，不能作为“欠载 FIFO 缺陷”的证据。

因此，要同时看 **V (读取批量)**、**C (可等多久)**、**逐盘分布**、**请求顺序和释放时刻**。只说“有长流和短流”，条件还不够。

容量取自此前 32 NPU / 6 SSU 实验配置；每卡接收链路为 50 GiB/s，也会影响到齐时间。数据算术见 [2]；闭环行为见 [3][4]。

有限请求：窗口选法也会改变答案

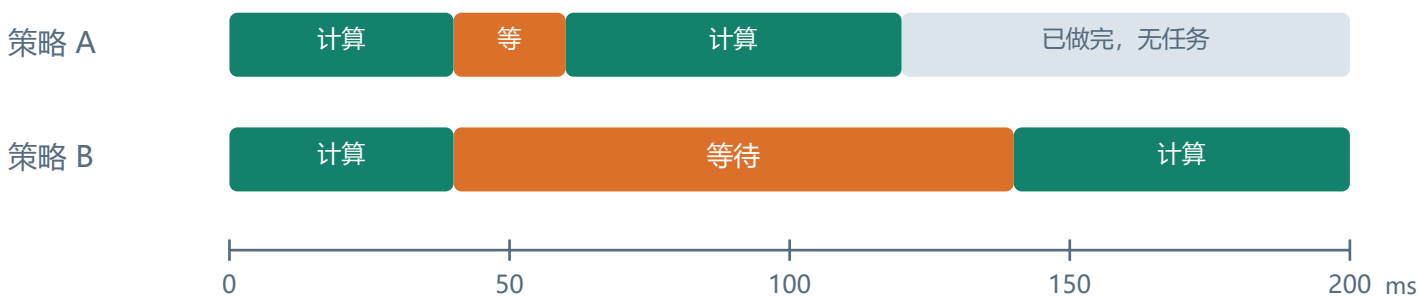
warm 是先让系统运行一段时间再统计；它本身不保证已经进入稳定状态。

之前的主统计窗口是 [2 s, 4 s)，共 2 秒。

32 卡平均利用率 = 窗口内 32 卡计算时间之和 / 64 卡秒

每张卡都有任务时，剩下的时间主要体现等待；队列耗尽后，还会混入无任务空闲。

教学例子：同一张卡、同一批任务，总计算都是 100 ms



都统计 0 ~ 200 ms：A 与 B 都是 $100 / 200 = 50\%$ 。

但完成时间不同：A 在 120 ms 做完，B 在 200 ms 做完。按各自完成时间算，分别是 83.3% 和 50%。

所以，扩大窗口时要同时报告：每卡是否还有任务、I/O 等待多少、总完成时间。混合请求还可能逐渐错开；一次突发造成等待，不代表以后永远这样。

miss 改变后，也不要只比利用率。命中更多，真正需要的计算减少，即便利用率变低，请求仍可能更早完成。判断策略时应固定输入，并一起看请求延迟、SLO 达标率与吞吐。

本页时间线为教学算例。之前的 TTFT SLO 口径是“接纳后到完成”，不包含接纳前排队；不能直接等同于用户从到达开始等待首 token 的时间。

读分析时，用这张对照表

先核实假设，再看公式；单次现象、长期结论与真实实验结果需要分开。

看到的说法	应该怎样理解
miss 长度 / miss 比例	长度 $M = pL$ ；同为 10% miss，总长不同，计算量也不同。
计算正比 miss 长度	固定条件下的近似；实际 C 查 data，不强行套同一常数。
带宽决定利用率	稳定服务的简化模型可估算；FIFO 还要看数据到齐时间。
开环 / 闭环	下一次释放是否依赖完成反馈；必须说明在讨论哪一层。
有限请求	任务清单会耗尽；扩大窗口可能混入结束后的空闲。

真正要问的是：短卡下一层的数据，能不能赶在当前计算结束前到齐？一条 FIFO 队列可能让答案变成“不能”，即使 V/C 的名义需求尚未用满盘。

核实来源（点击可打开）

[1] vLLM: Automatic Prefix Caching

命中前缀可复用 KV，跳过该部分的重复计算。

[2] 本项目 data 与 token 划分

第 2 页取 data 的 32K 四行；sim.py:231 定义计算与读取 token。

[3] 本项目 continuous_batch_sim.py

784 行读取 C；3156 行接纳；3269 行逐层预取；6672 行外部到达。

[4] Grafana k6: Open and closed models

开环与闭环到达模型、系统变慢对闭环发送速率的影响。

核对日期：2026-09-10。源码版本：e56b1f95bca6。本次仅核对代码与 data、制作教学图，没有重跑仿真。